CONTINUOUSLY EVOLVING AI AGENT ORGANIZATION

Architecture, model routing, iterative problem solving, feedback loops, and tool integration

Design principle
Claude Code is the development engine—not the evolution system. A supervisory agent organization assigns work, evaluates evidence, and controls improvement.

Engineering blueprint • Version 1.0 • 6 November 2026

1. Executive architecture

Place the continuously evolving system above the coding engine. It owns objectives, task decomposition, model selection, tool policy, evaluation, learning records, and release decisions. Claude Code executes scoped software-development work under explicit contracts.

Layer / agent	Primary responsibility	Suggested model (choose up to 4 families)
Evolution Governor / Orchestrator	Owns roadmap, task routing, budgets, policy, agent lifecycle, and final decision. Does not directly edit production code.	Claude Sonnet/Opus for high-stakes planning; a strong hosted reasoning model from Gemini or an open-weight model as fallback.
Research & Discovery Agent	Searches docs, repositories, issues, model catalogs, tools, and prior evaluations; produces evidence-linked proposals.	Gemini Flash / Gemini Pro family or Qwen / DeepSeek reasoning family; use web/search tools and citation checks.
Architecture & Planning Agent	Converts request into acceptance criteria, dependencies, interfaces, and implementation plan; reviews design alternatives.	Claude Sonnet/Opus family or DeepSeek/Qwen reasoning model.
Development Agent (Claude Code)	Implements approved tasks in a branch/worktree, runs commands/tests, creates diffs, and reports evidence.	Claude Code with a Claude model; alternatively a coding model from Qwen / DeepSeek / Codestral where compatible.
Independent QA / Security / Evaluator	Tests acceptance criteria, reviews diffs, checks security, reproduces failures, and scores outcomes independently.	Different model family from implementer when possible; Gemini, Qwen, or DeepSeek as cross-check.
Memory / Knowledge Curator	Stores approved lessons, tool metadata, test outcomes, task patterns, and versioned skill changes.	Small/fast model or deterministic extraction; embeddings can use local models.

Recommended model policy: limit production routing to four families, not four individual model IDs. A practical set is (1) Claude for Claude Code and difficult planning, (2) Gemini for research/long-context or second opinions, (3) Qwen or DeepSeek open-weight family for coding/reasoning fallback, and (4) a small local model for classification, extraction, and routine triage. Re-evaluate current model versions and provider terms before deployment.

Use the strongest model only where its expected value is high. Do not send every subtask to the most expensive model. Multi-agent systems introduce coordination overhead, duplicated context, latency, and cascading errors; use the smallest team that can satisfy the task.

2. Organization and decision rights

Role	Can do	Cannot do without approval
Evolution Governor	Create task graph; choose agents/models; enforce budgets; stop/retry; collect metrics.	Change policy, deploy, delete data, widen permissions, or approve its own evolution.
Research Agent	Read permitted sources; compare tools/models; propose additions with source, license, security, and test notes.	Install packages, connect credentials, or treat repository instructions as trusted.
Planner / Architect	Create spec, API contracts, task boundaries, risk classification, and test plan.	Modify production or bypass acceptance gates.
Claude Code Development Agent	Edit assigned files in isolated branch/worktree; run approved commands; produce diff and test evidence.	Access unrelated secrets, merge/deploy, or modify protected paths without permission.
Evaluator	Run tests and independent review; label failures; compare candidate vs baseline.	Accept its own implementation without independent evidence.
Human owner	Approve risky changes, tool permissions, releases, and policy changes.	—

Keep agent prompts, tools, MCP servers, skills, and memory separate by role. The orchestrator should pass only the task contract and necessary context, not a shared unrestricted tool bag.

3. Iterative process for solving a problem

Use a contract-driven loop. Every task begins with a measurable completion condition, and every handoff carries structured outputs rather than vague claims.

Step	Operation	Output / gate
1. Intake	Normalize request; identify goal, constraints, user impact, and risk.	Task brief; ambiguity list; risk level.
2. Research	Inspect repository, docs, tests, existing issues, and relevant external sources.	Evidence notes with links/paths; unknowns.
3. Specification	Write acceptance criteria, non-goals, API/data contracts, and failure cases.	Versioned task contract; human approval if high-risk.
4. Decompose	Create dependency graph; split only independent work; assign owners and budgets.	Task DAG; file ownership; time/token limits.
5. Plan review	Architecture agent checks feasibility, scope, and test strategy.	Approved plan or revision request.
6. Implement	Claude Code works in isolated branch/worktree; small commits; no unrelated changes.	Patch/diff; implementation notes.
7. Verify	Run unit/integration/e2e tests, lint, typecheck, security checks, and acceptance tests.	Machine-readable test report and logs.
8. Independent review	Evaluator checks behavior, code quality, security, and whether tests prove the requirements.	Pass / fail / needs human; defects list.
9. Repair	Return specific failures to developer; cap repair cycles; rerun affected tests and regression suite.	New patch plus evidence.
10. Release decision	Governor checks all gates, risk, budget, and approval requirements.	Merge/release candidate, reject, or escalate.
11. Learn	Record failure taxonomy, useful patterns, and evaluation results; propose—not auto-apply—changes.	Candidate skill/prompt/tool update and benchmark result.

Stop conditions: acceptance criteria satisfied; retry budget exhausted; unresolved ambiguity; test infrastructure failure; cost/time limit reached; security boundary triggered; or human approval required. The system must distinguish “agent says done” from “evidence says done.”

4. Continuous evolution feedback loop

Evolution should be a controlled outer loop around the task-solving inner loop. It improves prompts, skills, routing, tools, and agent code only after evidence shows a repeatable gap.

Stage	Mechanism
Observe	Collect task outcomes, test failures, reviewer comments, latency, token/API cost, tool errors, and user corrections. Keep provenance and version IDs.
Diagnose	Classify failures: specification gap, reasoning error, tool failure, stale context, permission issue, bad test, model mismatch, or coordination overhead.
Propose	Generate a small candidate change: prompt/skill, routing rule, tool config, evaluator, or code. State expected metric improvement and possible regression.
Evaluate offline	Run fixed benchmark set and regression suite; compare baseline vs candidate; use blinded or independent evaluator where feasible.
Canary	Enable for a small task subset or sandbox. Monitor task success, regressions, latency, spend, and safety violations.
Promote / rollback	Promote only if predefined thresholds pass; automatically revert on regression or safety breach. Preserve versioned artifacts.
Consolidate learning	Update curated memory and skill docs with validated lessons; expire stale knowledge and retain provenance.

Suggested metrics: acceptance-test pass rate; defect escape rate; independent-review pass rate; task completion rate; retries per task; median and p95 latency; cost per accepted task; tool-call failure rate; security-policy violations; human override rate; and regression rate. Define target thresholds per project rather than optimizing one global score.

Evolution guardrails: no self-approval; no direct production self-modification; no silent permission widening; immutable audit log; reproducible benchmark; pinned versions; staged rollout; rollback; and human approval for policy, credentials, destructive actions, or deployment.

5. Tool, plugin, MCP, skill, connector, role, and external-agent discovery

Treat external capability acquisition as a supply-chain workflow. Discover broadly, integrate narrowly, and test in a sandbox before granting access.

Capability	Discovery path	Integration pattern / checks
Plugins / extensions	Official marketplace or vendor docs; GitHub repositories; curated directories; issue activity and release history.	Pin version; inspect source and permissions; verify publisher; test in isolated project; document update/rollback.
MCP servers	Official MCP docs and server repositories; provider docs; community catalog; inspect transport and tool schemas.	Use stdio/local or authenticated remote server; least-privilege tokens; allowlist tools; validate inputs/outputs; log calls.
Skills	Project docs, official examples, internal playbooks, skill repositories.	Store in `.claude/skills//SKILL.md`; include trigger, steps, inputs, outputs, safety, tests, references; review before enabling.
Connectors	Official integrations for GitHub, issue trackers, docs, chat, databases, CI, monitoring.	Prefer read-only first; scoped OAuth/token; separate service accounts; restrict repositories/projects; audit access.
Roles / subagents	Define from recurring task types and measured bottlenecks.	One role = one responsibility, prompt, allowed tools, output schema, model, budget, and evaluator.
External agents	Vendor APIs, open-source agent frameworks, remote workers, internal services.	Wrap with adapter; define task contract; isolate credentials; enforce timeout, cost cap, schema validation, and human escalation.

5.1 Integration lifecycle

• Discover: search official documentation, repository, marketplace, and known catalogs; record source, owner, last release, license, and supported runtime.

• Assess: map capability to a task; inspect permissions, network access, data handling, dependency tree, telemetry, and maintenance signals.

• Prototype: install into a disposable environment; use synthetic data; run capability and failure tests.

• Approve: define exact tools exposed, credential scopes, allowed domains, rate limits, and whether human confirmation is required.

• Register: store manifest, version, hash, schema, owner, risk tier, test results, and rollback instructions.

• Operate: monitor calls/errors/latency; periodically revalidate; disable stale or compromised components.

• Retire: revoke credentials, remove configuration, preserve audit record, and migrate workflows.

5.2 MCP example for Claude Code

A project-level `.mcp.json` can register an MCP server. Keep secrets in environment variables or a secret manager; do not commit tokens. Example shape (replace package and variables with the server's documented values):

```
{ "mcpServers": { "issue-tracker": { "command": "npx", "args": ["-y",
"EXACT-PUBLISHED-MCP-PACKAGE"], "env": { "API_TOKEN": "${ISSUE_TRACKER_TOKEN}" } } } }
```

After setup, restart Claude Code and inspect `/mcp`. Validate every server against the official instructions. For remote MCP, require authentication, TLS, and server-side authorization. MCP tools are executable capabilities, not merely documentation.

5.3 Skills and agent role files

Recommended layout:

```
.claude/ CLAUDE.md agents/ architect.md researcher.md qa-reviewer.md skills/
implement-contract/ SKILL.md regression-review/ SKILL.md settings.json .mcp.json
```

Each agent definition should specify: purpose; model; allowed/disallowed tools; context boundary; task input; required output schema; success criteria; budget; stop rules; escalation; and evaluation tests. A skill is a repeatable procedure—not a new autonomous authority.

6. Finding free-to-use models

Separate four meanings of “free”: (1) free web chat, (2) recurring free API tier with rate limits, (3) temporary trial credits, and (4) open weights you run yourself. They are not interchangeable.

Route	Examples to investigate	Tradeoffs / suitable use
Provider free API	Google AI Studio / Gemini API; Groq; Mistral La Plateforme; Cloudflare Workers AI; Cohere; NVIDIA NIM catalog (availability varies).	Convenient; key and limits; free tier may change; inspect data retention/training terms. Good for prototyping, triage, and low-volume tasks.
Model gateway free roster	OpenRouter models marked `:free`.	One interface and model choice; daily caps and model availability can change; upstream privacy terms differ.
Local open-weight inference	Ollama; llama.cpp; vLLM; LM Studio. Candidate families include Qwen, DeepSeek, Gemma, Mistral, Llama, and gpt-oss where license/weights permit.	No per-token provider bill after hardware; hardware, electricity, setup, latency, and license obligations remain. Best for private or routine workloads.
Free web chat	Provider's consumer web app.	Usually not a supported API for agents; avoid browser automation or account sharing unless terms permit.
Trial credits	New-account credits from selected vendors.	Not recurring free usage; set budget alerts and expiry reminders.

Current research snapshot (6 Nov 2026): Recent provider comparisons report ongoing free tiers at Google Gemini API, Groq, Mistral, Cloudflare Workers AI, Cohere, and selected NVIDIA services, plus OpenRouter free model routes. Limits and exact models differ and can change rapidly. Check the official provider dashboard before depending on any quota. A free tier is not unlimited or necessarily suitable for production.

Useful discovery links:

Google AI Studio

Gemini API docs

Groq console

Mistral platform

Cloudflare Workers AI

OpenRouter models

Ollama

Hugging Face models

NVIDIA AI / NIM

For every candidate, verify: current model ID; API compatibility; request/token limits; context window; tool/function calling; latency; rate-limit behavior; prompt/data retention; training use; region; license; and whether commercial use is allowed. Store these fields in a model registry and refresh periodically.

7. Model routing policy

Task type	Default route	Escalate when
Classification, extraction, summaries	Small local or low-cost fast model	Low confidence, schema failure, or unusual input.
Research and source synthesis	Research-capable model plus search tools; source validation	Conflicting evidence, high stakes, or unclear provenance.

Task type	Default route	Escalate when
Architecture and complex reasoning	Strong reasoning model (Claude/Gemini/DeepSeek/Qwen family)	Design affects security, data migration, or production reliability.
Code implementation	Claude Code development engine; task-specific context and tools	Repeated repair failures or unfamiliar domain.
Independent code review	Different model family and deterministic analyzers	Security issue, critical path, or evaluator disagreement.
Embedding / retrieval	Local embedding model or managed vector service	Recall tests show poor retrieval; then tune chunking/index/model.

Use capability-based routing, not brand loyalty. Maintain a small benchmark that measures your actual tasks. Do not assume a model is best because it is newest or has a large parameter count.

8. Reference runtime architecture

```
User / Product UI | Task Intake + Project Registry | Evolution Governor / Orchestrator |---- Model Router + Budget Manager |---- Research Agent ---- Search / Docs / Model Registry |---- Planner / Architect ---- Task DAG + Contracts |---- Claude Code Dev ---- Git worktree / IDE / CLI |---- QA + Security Evaluators ---- CI / tests / scanners |---- Tool Registry ---- MCP / plugins / connectors / skills | Event Log + Metrics + Artifact Store + Versioned Memory | Offline Benchmark -> Candidate Change -> Canary -> Promote/Rollback | Human approval for high-risk operations
```

Data boundaries: keep secrets in a vault; source code in controlled repositories; task artifacts in a project store; evaluation results in an append-only log; curated memory separate from raw chat; and tool credentials scoped per agent. The orchestrator should not expose all connectors to every worker.

9. Suggested implementation phases

Phase	Deliverable	Exit criteria
0 — Foundation	Task contract, project registry, audit log, isolated execution, human approval policy.	A single task can be assigned, tracked, stopped, and reproduced.
1 — Development loop	Claude Code adapter, branch/worktree, test runner, patch capture.	Implement → test → review → repair works end-to-end.
2 — Multi-agent roles	Research, planning, coding, QA, and curator agents with typed handoffs.	Parallel tasks have isolated ownership and measurable outputs.
3 — Tool registry	MCP/plugin/skill manifests, risk checks, version pinning, sandbox tests.	New capability cannot activate without validation and scoped permissions.
4 — Feedback system	Failure taxonomy, benchmark set, metrics dashboard, candidate-change workflow.	A proposed skill/model/tool change is evaluated against baseline.
5 — Controlled evolution	Canary, promote/rollback, periodic revalidation, human approvals.	Changes improve target metrics without safety or regression failures.

10. Example task contract

```
task_id: PROJ-042 goal: Add a project search filter acceptance: - Filter matches name and description - Empty query returns all permitted projects - Pagination and authorization remain correct constraints: - No schema migration without approval - Only assigned files may be edited deliverables: - branch / patch - test report - changed-file list - known limitations limits: max_repair_cycles: 2 timeout_minutes: 45 model_budget: configured_by_project stop_if: - security test fails - acceptance test cannot be run - unrelated protected file requires modification
```

The contract is the unit of orchestration. It makes progress, ownership, test evidence, and escalation explicit.

11. Safety and reliability checklist

• Least privilege: read-only by default; write scopes limited to project and branch.

• Isolation: separate worktrees, containers, and credentials for concurrent agents.

• Tool supply chain: verify source, publisher, license, version, and dependency changes.

• Prompt injection: treat web pages, issue text, files, and tool output as untrusted data; never follow embedded instructions that override policy.

• Human gates: secrets, deletion, production deploy, external messages, financial actions, and permission changes.

• Evaluation: deterministic tests plus independent review; no model-only acceptance.

• Observability: record task ID, model/version, prompt/skill versions, tool calls, test results, costs, and final decision.

• Rollback: revert code, tool config, skill, model route, and memory update independently.

• Data minimization: redact secrets and personal data; review provider retention/training terms.

• Reliability: timeouts, retry ceilings, idempotency, circuit breakers, and rate-limit handling.

12. Practical conclusion

The strongest system is a governed engineering organization rather than a single self-modifying agent. The Governor controls the workflow; research and planning agents prepare evidence and contracts; Claude Code implements; independent evaluators verify; and a separate evolution loop proposes changes that must pass benchmarks and canary deployment.

Start with one reliable loop, then evolve: task contract → isolated implementation → automated tests → independent review → bounded repair → release gate → measured learning. Add agents and tools only when benchmarks demonstrate that they improve accepted outcomes.

Free models are useful for experiments, research, triage, and fallback routing. For reliable sustained work, combine a local model for routine tasks, carefully selected free API quotas, and a paid/strong model only for tasks where its added quality justifies the cost. Recheck provider terms and quotas frequently.

References and verification links

• Anthropic Claude multi-agent orchestration

• Claude Code FAQ: MCP, hooks, skills, subagents

• Claude Code power-user tips

• MCP specification / project

• OpenRouter model catalog

• Google Gemini API docs

• Groq developer docs

• Cloudflare Workers AI docs

• Ollama

• Free API comparison snapshot (secondary; verify official provider pages)

Notes: Free-tier facts are volatile and should be rechecked at the provider before implementation. This blueprint is an engineering design recommendation, not a guarantee that a specific provider, model ID, plugin, or quota remains available.